Gałęzie w gicie

Jak pracujemy w quiz-projekt

Ściąga dla nas obu · stan repozytorium z 3 listopada 2026

W skrócie

1. Na stałe są **tylko dwie gałęzie**: `main` i `deploy`.
2. Każde zadanie robisz na **nowej gałęzi** odbitej od `deploy`, np. `feature/front-logowanie`.
3. Do `deploy` i `main` zmiany trafiają **tylko przez pull request** na GitHubie, nigdy bezpośrednim commitem.
4. Po scaleniu PR **gałąź zadania usuwasz**. Praca nie znika: commity zostają w `deploy`.
5. Przed nowym zadaniem zawsze: `git switch deploy` i `git pull`.

1 Podstawy

- **Commit** to zapisany stan projektu z opisem, np. „Dodanie formularza logowania". Historia projektu to ciąg commitów.
- **Gałąź (branch)** to osobna linia commitów. Na swojej gałęzi zmieniasz, co chcesz, i nie psujesz pracy kolegi. Gałąź to tylko nazwa wskazująca na ostatni commit, dlatego usunięcie scalonej gałęzi niczego nie kasuje.
- **Merge** to połączenie zmian z jednej gałęzi do drugiej.
- **Pull request (PR)** to prośba na GitHubie: „wciągnij zmiany z mojej gałęzi do `deploy`". Widać w nim każdą zmienioną linię, drugi z nas może to przejrzeć, skomentować i kliknąć *Merge*.

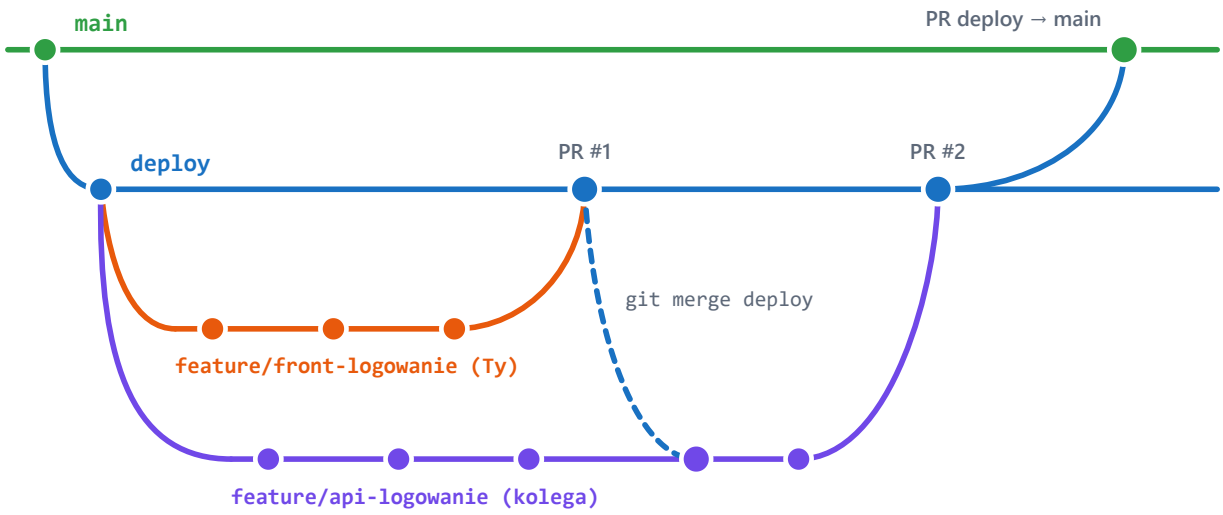
Dwie kopie repozytorium

Repo jest u Ciebie na komputerze i na GitHubie (git nazywa GitHuba `origin`). Te dwie kopie same się nie synchronizują:

POLECENIE	CO ROBI
`git push`	wysyła Twoje commity na GitHuba
`git pull`	pobiera z GitHuba nowe commity do gałęzi, na której jesteś
`git fetch`	pobiera tylko informację, co nowego na GitHubie (np. nowe gałęzie kolegi); Twoich plików nie zmienia

2 Nasze gałęzie

GAŁĄŹ	DO CZEGO	KTO TU COMMITUJE	JAK DŁUGO ŻYJE
`main`	wersja gotowa – to pokazujemy na obronie	nikt bezpośrednio, tylko PR z `deploy`	zawsze
`deploy`	wersja robocza – tu zbieramy skończone zadania	nikt bezpośrednio, tylko PR z gałęzi zadań	zawsze
`feature/…`	jedna nowa rzecz, np. `feature/front-logowanie`	osoba, która robi to zadanie	do scalenia PR
`fix/…`	poprawka błędu, np. `fix/zle-haslo`	osoba, która poprawia	do scalenia PR



Każda kropka to commit. Pracujemy równolegle, każdy na swojej gałęzi. Gotowe zadanie trafia do `deploy` przez PR (#1). Kolega w trakcie pracy dociąga do siebie nowe `deploy` (przerywana linia), kończy i robi PR (#2). Gdy `deploy` działa, robimy PR z `deploy` do `main`.

Nazwa `deploy` jest u nas umowna. W wielu projektach taką gałąź roboczą nazywa się `develop`.

Jak nazywać gałęzie

- na początku `feature/` (nowa rzecz) albo `fix/` (poprawka błędu);
- dalej `front-` albo `api-`, żeby od razu było widać, czy to frontend, czy backend;
- małe litery, słowa oddzielone myślnikiem, **bez spacji i polskich znaków**;
- przykłady: `feature/front-strona-glowna`, `feature/api-quizy`, `fix/api-zle-haslo`.

3 Nowe zadanie krok po kroku

1 **Przejdź na `deploy` i pobierz najnowszą wersję.**

```
git switch deploy
git pull
```

2. **Utwórz gałąź zadania i od razu na nią przejdź.**

```
git switch -c feature/front-logowanie
```

3. **Pracuj i zapisuj zmiany.** Commituj często, małymi kawałkami, z opisem, co zrobiłeś.

```
git status                              # co się zmieniło i na jakiej gałęzi jesteś
git add .
git commit -m "Dodanie formularza logowania"
```

4. **Wyślij gałąź na GitHuba.** Za pierwszym razem z `-u` (podpina gałąź pod GitHuba), potem wystarczy samo `git push`.

```
git push -u origin feature/front-logowanie
```

5. **Otwórz pull request na GitHubie** – opis niżej, w punkcie 4.
6. **Po scaleniu PR posprzątaj u siebie.**

```
git switch deploy
git pull
git branch -d feature/front-logowanie
```

4 Pull request na GitHubie

1. Po `git push` na stronie repozytorium pojawi się żółty pasek z przyciskiem **Compare & pull request**. Kliknij go. Jeśli paska nie ma: zakładka **Pull requests** → **New pull request**.
2. U góry ustaw, skąd i dokąd idą zmiany. **base** to gałąź, *do której* trafią zmiany: `base: deploy` ← `compare: feature/front-logowanie`
3. Wpisz tytuł i krótko opisz, co zrobiłeś. Kliknij **Create pull request**.
4. Drugi z nas otwiera PR. Zakładka **Files changed** pokazuje wszystkie zmiany, a przy każdej linii można dodać komentarz.
5. Gdy wszystko jest dobrze: **Merge pull request** → **Confirm merge**.
6. Kliknij **Delete branch**. Gałąź znika z GitHuba, a commity zostają w `deploy`.

> **WYDANIE DO MAIN**
>
> Gdy na `deploy` wszystko działa (np. przed oddaniem etapu albo przed obroną), robimy PR tak samo, tylko `base: main` ← `compare: deploy`. Gałęzi `deploy` po tym **nie usuwamy**.

> **KTO KLIKA MERGE**
>
> Swój PR scala druga osoba. Wtedy każdy z nas wie, co się zmieniło w projekcie, a błąd łatwiej wyłapać przed wrzuceniem do `deploy`.

5 Praca we dwóch

Podział pracy

Jeden robi w `frontend/`, drugi w `backend/`. Wtedy prawie nie ma konfliktów. Wspólne pliki (`docker-compose.yml`, `database/`, `docker/`) zmieniamy po uzgodnieniu.

Kolega scalił swoje do `deploy`, a ja chcę to mieć u siebie

Np. potrzebujesz jego nowego API. Dociągasz najnowsze `deploy` do swojej gałęzi i pracujesz dalej:

```
git switch deploy
git pull
git switch feature/front-logowanie
git merge deploy
```

Chcę zobaczyć gałąź kolegi, która nie jest jeszcze scalona

```
git fetch                                # pobiera informację o nowych gałęziach
git switch feature/api-logowanie         # sam tworzy lokalną kopię gałęzi z GitHuba
git switch -                             # powrót na poprzednią gałąź
```

6 Konflikty

Konflikt jest wtedy, gdy obaj zmieniliśmy **te same linie tego samego pliku** i git nie wie, którą wersję zostawić. Git zatrzymuje merge i zaznacza to miejsce w pliku:

```
<<<<<<< HEAD
<h1>Quizownia</h1>            ← Twoja wersja
=======
<h1>Nasze quizy</h1>          ← wersja z deploy
>>>>>>> deploy
```

1. `git status` pokaże pliki z konfliktem (*both modified*).
2. Otwórz plik. VS Code pokazuje nad konfliktem przyciski *Accept Current Change*, *Accept Incoming Change* i *Accept Both Changes*. Wybierz jeden albo popraw ręcznie. W pliku nie mogą zostać znaczniki `<<<<<<<`, `=======`, `>>>>>>>`.
3. Zapisz plik i dokończ merge:

```
git add .
git commit -m "Rozwiązanie konfliktu w index.html"
```

> **WOLISZ SIĘ WYCOFAĆ?**
>
> `git merge --abort` przywraca wszystko do stanu sprzed merge.

7 branch, switch i checkout

POLECENIE	CO ROBI
`git branch`	lista gałęzi, gwiazdka `*` oznacza tę, na której jesteś
`git branch nazwa`	tworzy gałąź, ale **zostajesz na obecnej**
`git branch -d nazwa`	usuwa gałąź; nie pozwoli usunąć niescalonej, więc nic nie zginie
`git switch nazwa`	przechodzi na gałąź – pliki w folderze zmieniają się na jej wersję
`git switch -c nazwa`	tworzy gałąź i od razu na nią przechodzi
`git switch -`	wraca na poprzednią gałąź
`git checkout nazwa`	starsze polecenie, to samo co `git switch nazwa`
`git checkout -b nazwa`	to samo co `git switch -c nazwa`

Używamy `git switch`, bo służy tylko do gałęzi. `git checkout` robi też inne rzeczy: np. `git checkout plik.php` po cichu kasuje niezapisane zmiany w tym pliku.

8 Zasady

TAK

- każde zadanie zaczynaj od `git switch deploy` i `git pull`
- jedna gałąź = jedno zadanie
- commituj często, z opisem, co zrobiłeś
- przed przełączeniem gałęzi zrób commit (sprawdź `git status`)
- PR scala druga osoba
- po scaleniu usuń gałąź zadania

NIE

- nie commituj prosto na `main` ani `deploy`
- nie usuwaj `main` ani `deploy`
- nie używaj `git push --force`
- nie wracaj do starej, scalonej gałęzi – zrób nową od `deploy`
- nie zmieniaj wspólnych plików bez uzgodnienia

9 Gdy coś pójdzie nie tak

Zacząłem pracę na `deploy` zamiast na nowej gałęzi (jeszcze bez commita)

Utwórz gałąź teraz. Niezapisane zmiany przejdą razem z Tobą na nową gałąź:

```
git switch -c feature/front-logowanie
```

Zrobiłem commit na `deploy`, ale jeszcze go nie wysłałem

Najpierw upewnij się, że `git status` nie pokazuje niezapisanych zmian. Potem:

```
git branch feature/front-logowanie        # nowa gałąź z Twoim commitem
git reset --hard origin/deploy            # deploy wraca do wersji z GitHuba
git switch feature/front-logowanie        # dalej pracujesz na swojej gałęzi
```

> **UWAGA**
>
> `git reset --hard` kasuje niezapisane zmiany. Jeśli commit jest już na GitHubie, nie rób tego – napisz do kolegi i ustalcie, co dalej.

`git push` kończy się błędem „rejected"

Ktoś wysłał coś na tę gałąź przed Tobą. Pobierz jego zmiany i wyślij jeszcze raz:

```
git pull
git push
```

Nie mogę przełączyć gałęzi, bo mam niezapisane zmiany

Zrób commit. Albo schowaj zmiany na chwilę i wyjmij je po powrocie:

```
git stash                                 # chowa zmiany
git switch deploy
git switch -                              # wracasz na swoją gałąź
git stash pop                             # zmiany wracają
```

Usunąłem gałąź z GitHuba przez pomyłkę

Otwórz jej pull request na GitHubie – jest tam przycisk **Restore branch**.

10 Ściąga poleceń

POLECENIE	CO ROBI
`git status`	na jakiej gałęzi jestem i co zmieniłem – wpisuj często
`git log --oneline --graph --all`	historia wszystkich gałęzi jako drzewko
`git branch`	lista gałęzi u Ciebie
`git switch deploy`	przejście na `deploy`
`git switch -c feature/nazwa`	nowa gałąź zadania
`git add .`	dodanie wszystkich zmian do następnego commita
`git commit -m "Opis"`	zapis zmian
`git push -u origin feature/nazwa`	pierwsze wysłanie nowej gałęzi na GitHuba
`git push`	kolejne wysłania
`git pull`	pobranie zmian z GitHuba do gałęzi, na której jesteś
`git fetch --prune`	odświeżenie listy gałęzi z GitHuba, usuwa z listy te skasowane
`git merge deploy`	dociągnięcie najnowszego `deploy` do Twojej gałęzi
`git merge --abort`	wycofanie merge z konfliktem
`git stash` / `git stash pop`	schowanie i wyjęcie niezapisanych zmian
`git branch -d feature/nazwa`	usunięcie gałęzi u siebie
`git push origin --delete feature/nazwa`	usunięcie gałęzi z GitHuba

11 Jednorazowe porządki

Stan na 3 listopada 2026. Po tych krokach na GitHubie zostaną tylko `main` i `deploy`, z tą samą zawartością.

Usunięcie starych gałęzi

GAŁĄŹ	DLACZEGO DO USUNIĘCIA
`main-page`	wszystko z niej jest już w `deploy` i `main`
`database-info`	stara, scalona do `main`
`fast_api_integration`	stary pomysł z FastAPI, scalony do `main`

```
git switch deploy
git pull
git branch -d main-page
git push origin --delete main-page database-info fast_api_integration
```

Drugi z nas robi u siebie `git fetch --prune`, żeby te gałęzie zniknęły też z jego listy.

Usunięcie `backend/requirements.txt` z `main`

Na `main` został plik po FastAPI, którego nie ma w `deploy`. Usuwamy go zwykłą drogą, przez gałąź i PR:

```
git switch deploy
git switch -c fix/porzadek
git merge main                          # dociąga to, co jest tylko w main
git rm backend/requirements.txt
git commit -m "Usunięcie requirements.txt po FastAPI"
git push -u origin fix/porzadek
```

Potem PR `fix/porzadek` → `deploy`, a na końcu PR `deploy` → `main`.

Ustawienia na GitHubie

Zmienia je właściciel repozytorium, wystarczy raz:

- **Settings** → **General** → **Pull Requests**: zaznacz *Automatically delete head branches*. Gałąź zadania sama zniknie z GitHuba po scaleniu PR.
- **Settings** → **Branches**: dodaj regułę dla `main` i dla `deploy` z opcją *Require a pull request before merging*. GitHub nie pozwoli wtedy wysłać commita prosto na te gałęzie. W prywatnym repozytorium na darmowym koncie ta opcja może być niedostępna – wtedy po prostu pilnujemy zasady sami.